

Introduction to Algorithm

기초 알고리즘

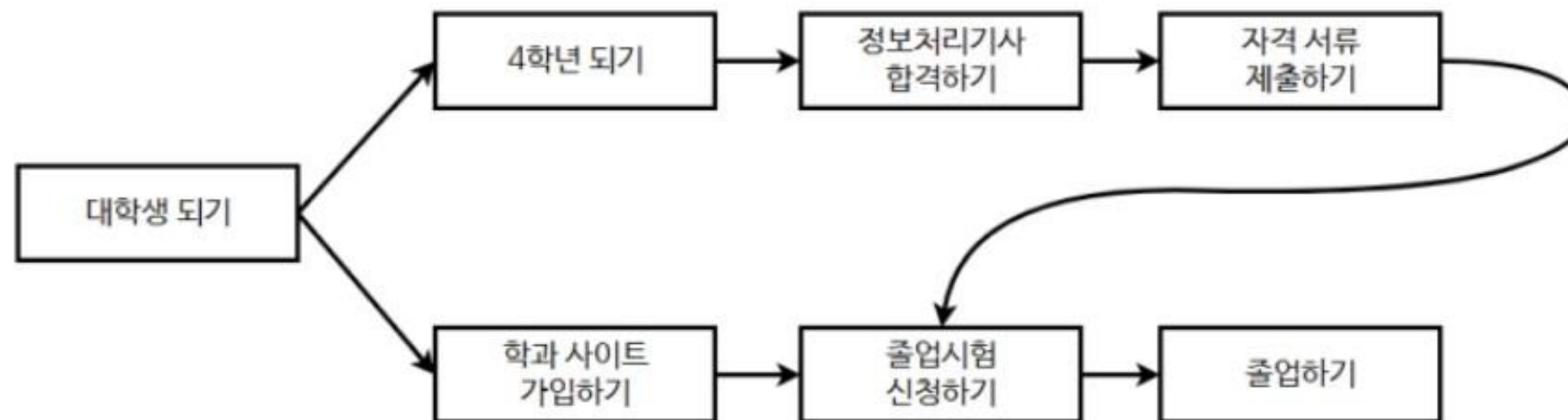
그래프·트리 알고리즘 응용

위상 정렬



위상 정렬 (Topological Sorting)

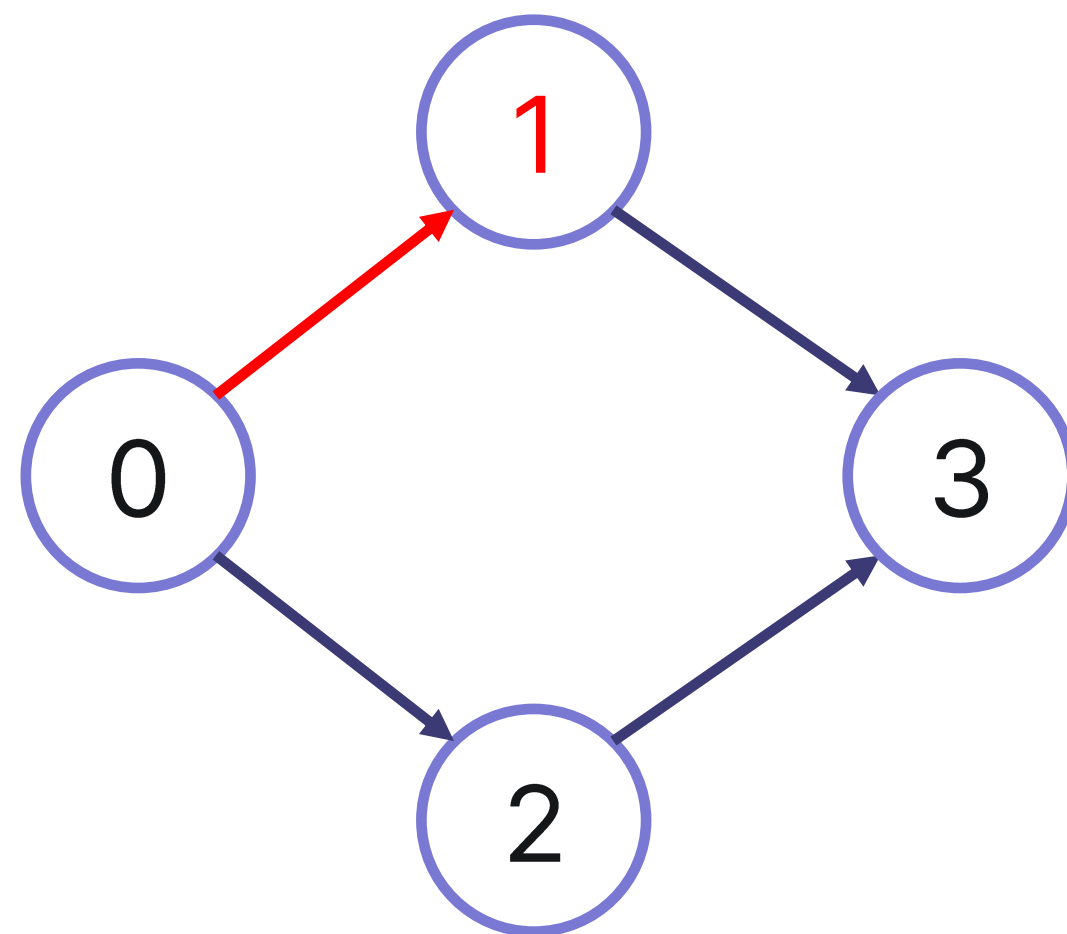
- 순서가 정해져 있는 작업을 차례로 수행해야 할 때, 그 순서를 결정해주기 위해 사용
- Directed Acyclic Graph (DAG) 에서만 적용 가능
 - DAG는 사이클이 없는 방향 그래프를 의미
 - 사이클이 있는 경우에는 위상 정렬이 불가





위상 정렬의 동작 방식

1. 진입 차수(In-degree)가 0인 정점을 큐에 삽입
 - 진입 차수: 다른 정점에서 들어오는 간선의 개수

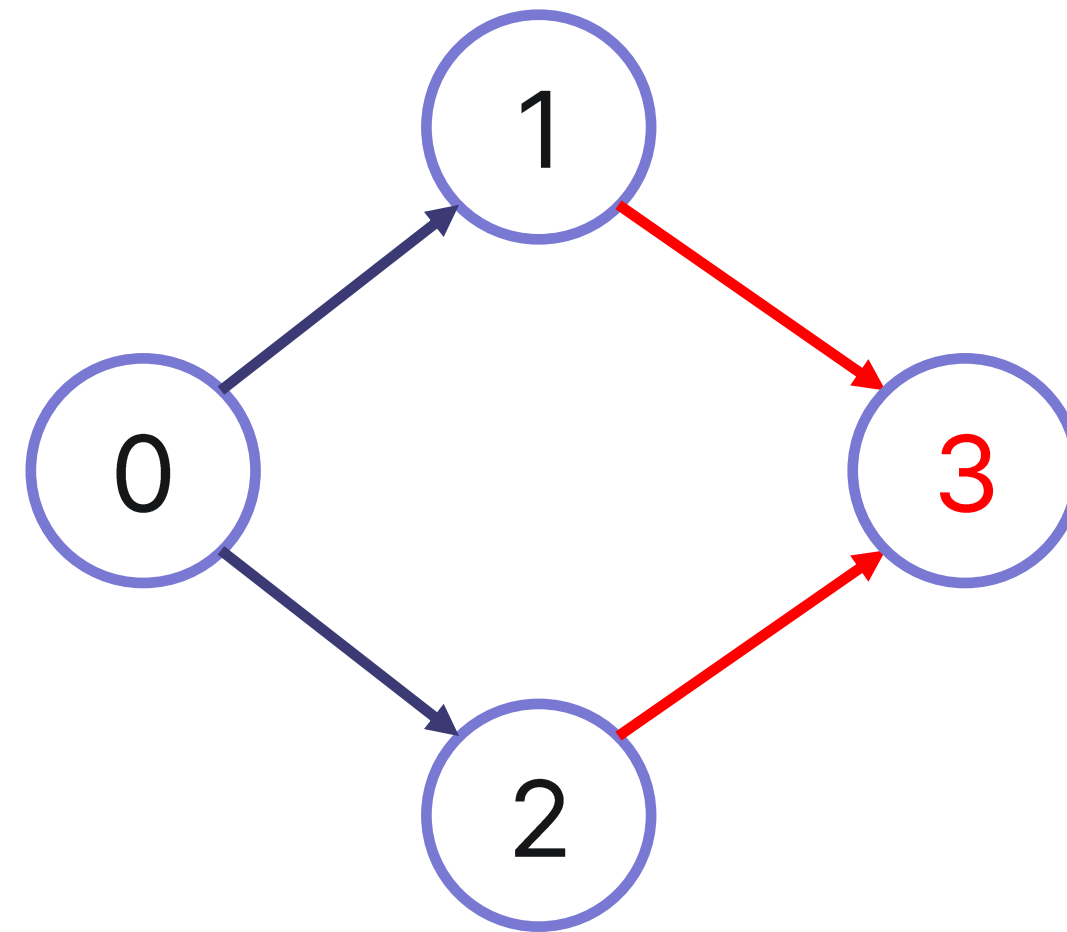


정점 '1'의 진입 차수: 1



위상 정렬의 동작 방식

1. 진입 차수(In-degree)가 0인 정점을 큐에 삽입
 - 진입 차수: 다른 정점에서 들어오는 간선의 개수

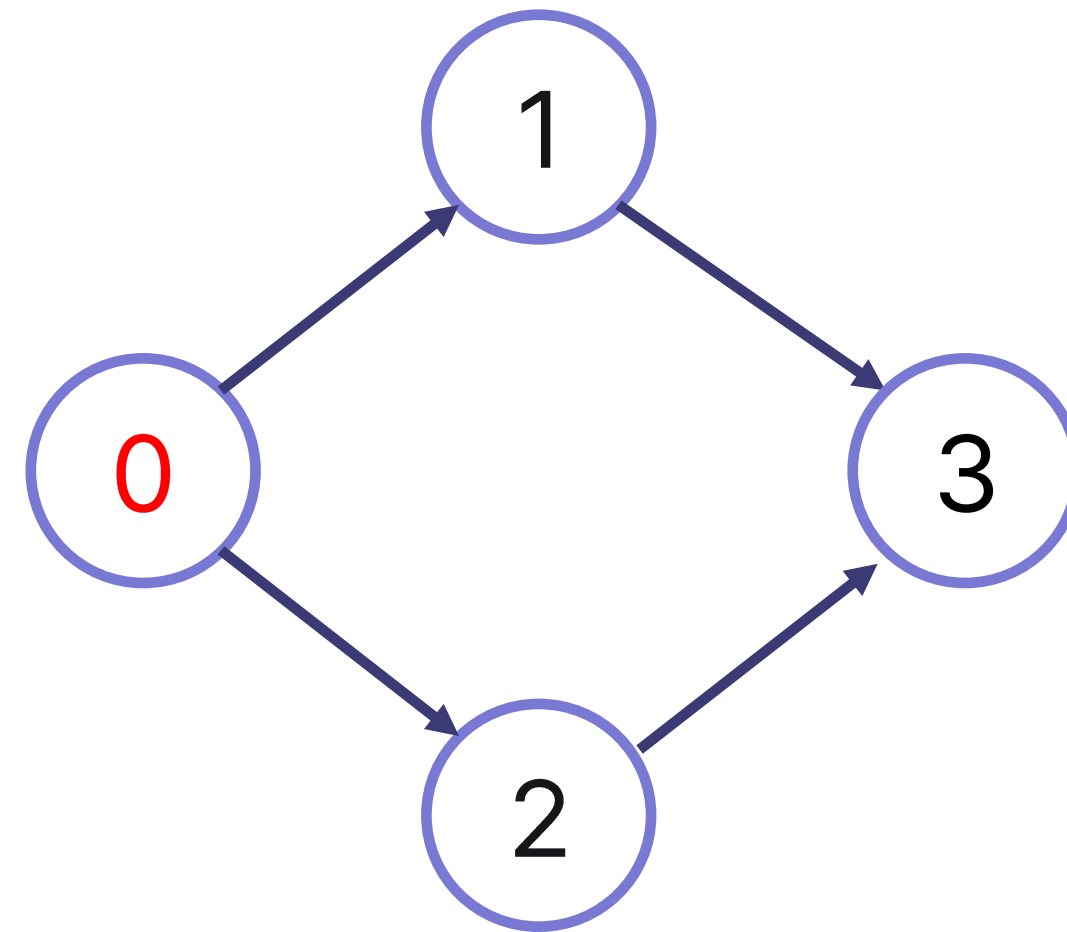


정점 '3'의 진입 차수: 2



위상 정렬의 동작 방식

1. 진입 차수(In-degree)가 0인 정점을 큐에 삽입
 - 진입 차수: 다른 정점에서 들어오는 간선의 개수



정점 '0'의 진입 차수: 0

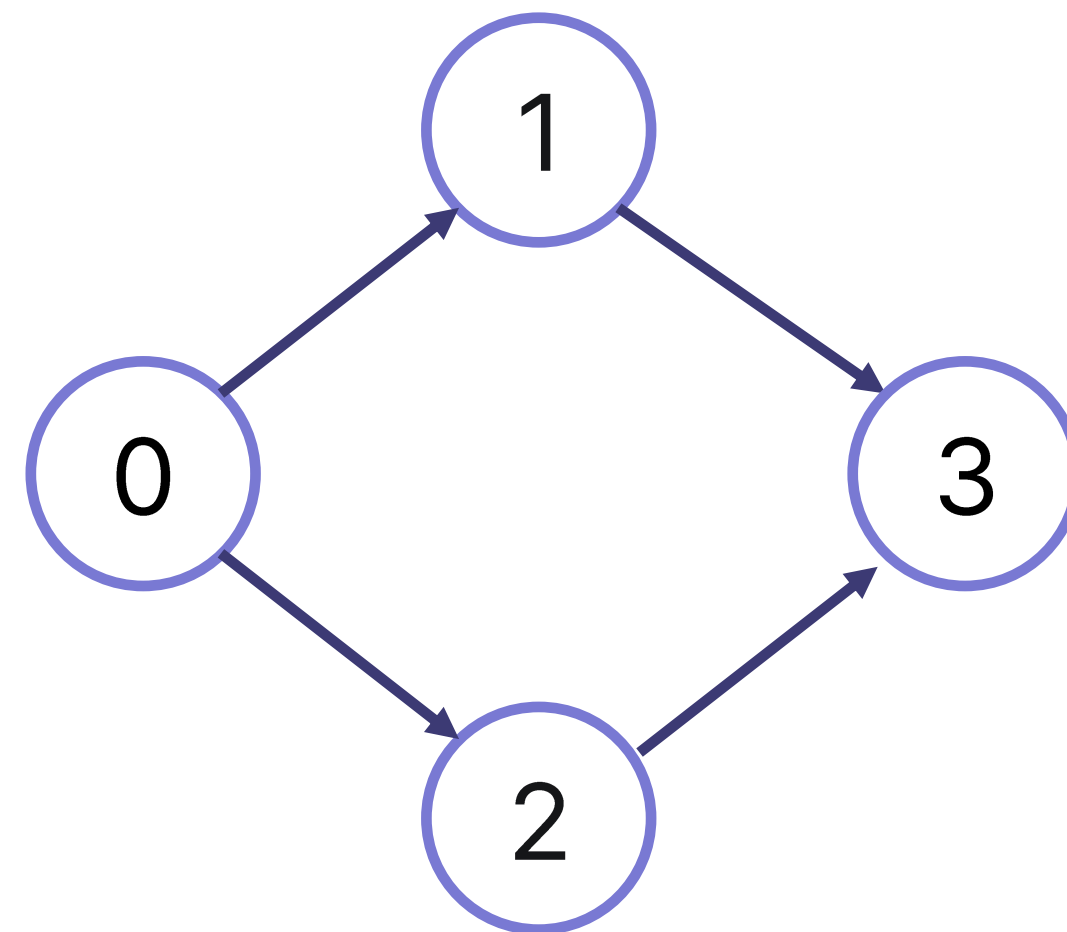
→ 큐: [0]



위상 정렬의 동작 방식

2. 큐에서 원소를 꺼내 연결된 모든 간선을 제거

- 큐에서 하나 꺼내 위상 정렬에 추가
- 해당 정점에서 나가는 간선 제거 (= 도착 정점의 진입 차수 감소)



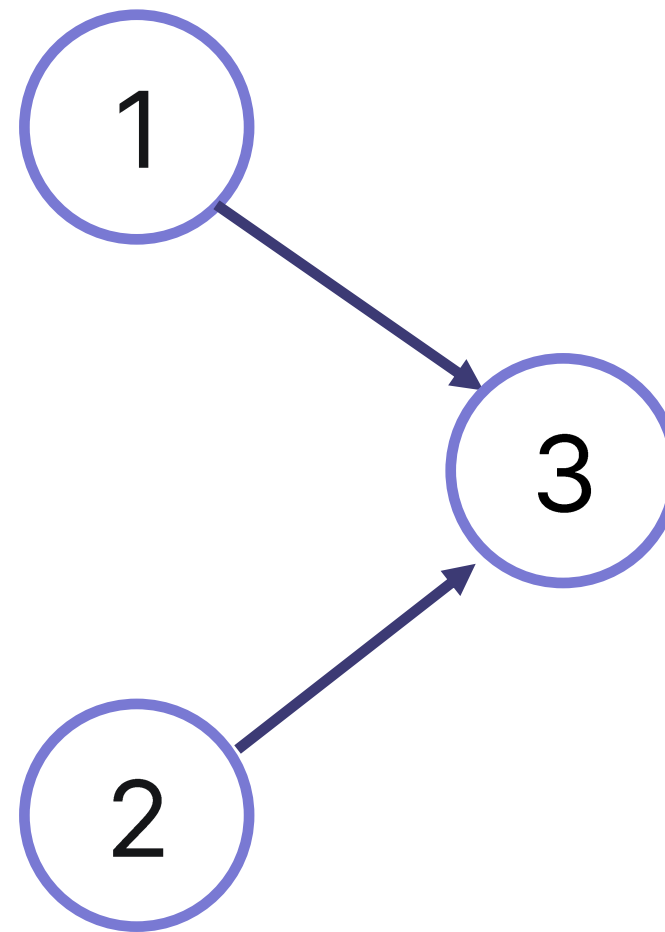
큐: [0]



위상 정렬의 동작 방식

2. 큐에서 원소를 꺼내 연결된 모든 간선을 제거

- 큐에서 하나 꺼내 위상 정렬에 추가
- 해당 정점에서 나가는 간선 제거 (= 도착 정점의 진입 차수 감소)

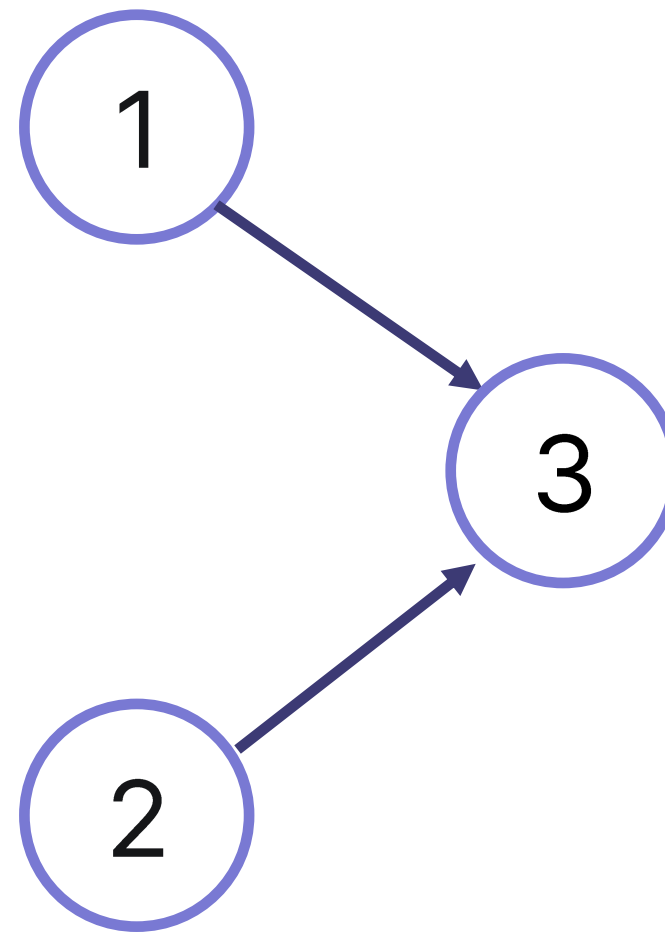


큐: []
위상 정렬: [0]



3. 큐가 빌 때까지 1~2번 과정을 반복

- 모든 원소를 방문하기 전에 큐가 빈다면, 사이클 존재
- 모든 원소를 방문했다면, 큐에서 꺼낸 순서가 위상 정렬의 결과

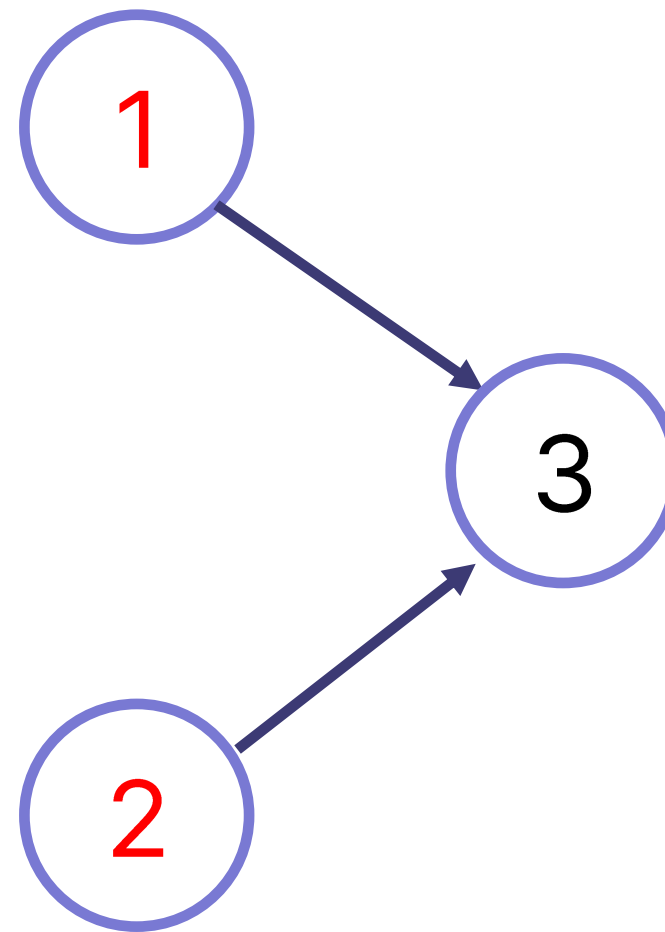


큐: []
위상 정렬: [0]



3. 큐가 빌 때까지 1~2번 과정을 반복

- 모든 원소를 방문하기 전에 큐가 빈다면, 사이클 존재
- 모든 원소를 방문했다면, 큐에서 꺼낸 순서가 위상 정렬의 결과



큐: [1, 2]
위상 정렬: [0]



3. 큐가 빌 때까지 1~2번 과정을 반복

- 모든 원소를 방문하기 전에 큐가 빈다면, 사이클 존재
- 모든 원소를 방문했다면, 큐에서 꺼낸 순서가 위상 정렬의 결과

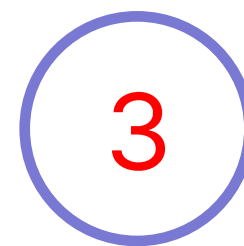
3

큐: []
위상 정렬: [0, 1, 2]



3. 큐가 빌 때까지 1~2번 과정을 반복

- 모든 원소를 방문하기 전에 큐가 빈다면, 사이클 존재
- 모든 원소를 방문했다면, 큐에서 꺼낸 순서가 위상 정렬의 결과



큐: [3]
위상 정렬: [0, 1, 2]



3. 큐가 빌 때까지 1~2번 과정을 반복

- 모든 원소를 방문하기 전에 큐가 빈다면, 사이클 존재
- 모든 원소를 방문했다면, 큐에서 꺼낸 순서가 위상 정렬의 결과

큐: []

위상 정렬: [0, 1, 2, 3]



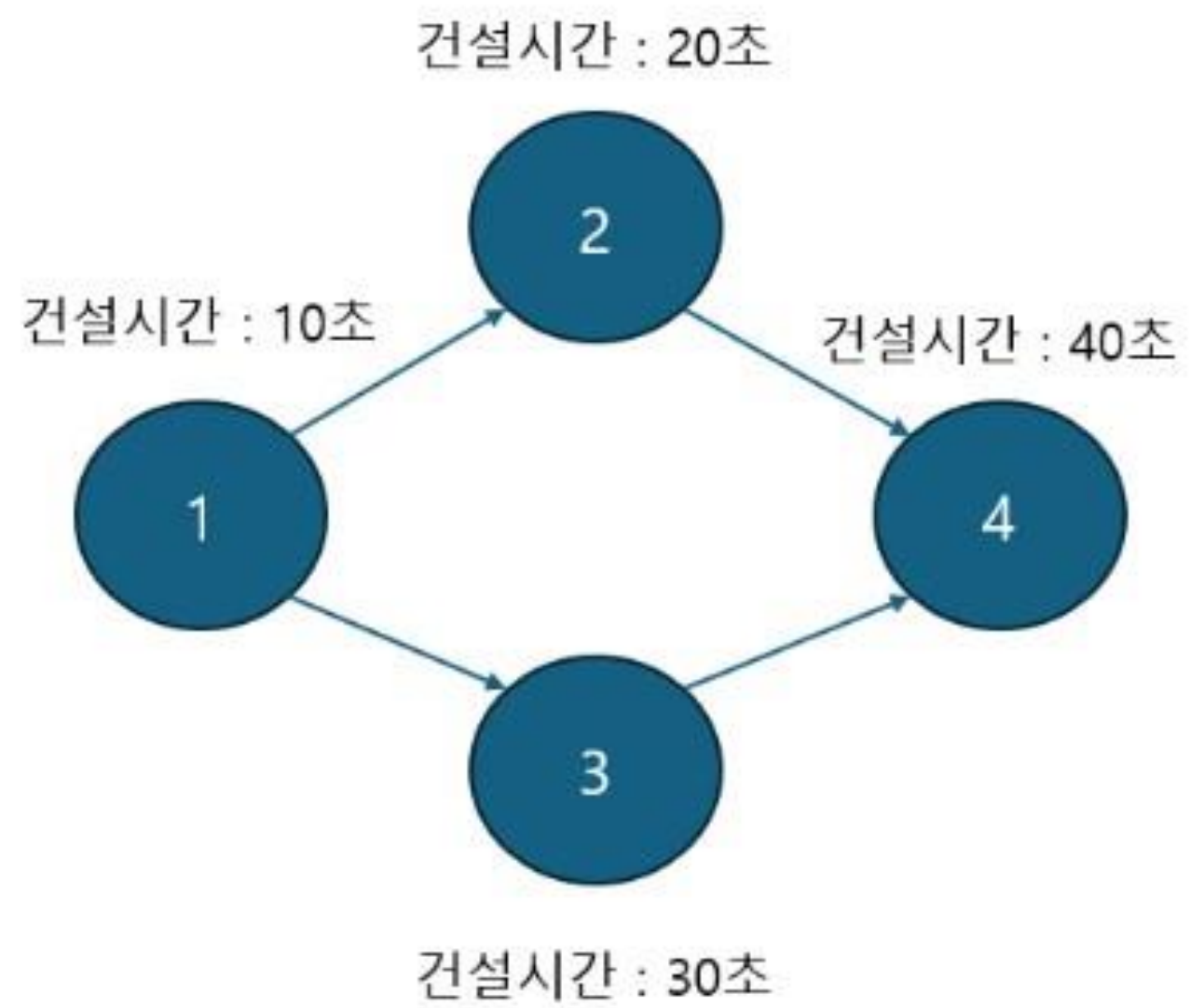
위상 정렬 예시

- 엘리스는 여러 개의 건물을 건설하는 프로젝트를 관리하고 있다.
- 각 건물은 건설을 시작하기 전에 반드시 먼저 완성되어야 하는 건물들이 존재할 수 있다.
- 각 건물은 건설 시간이 서로 다르며, 선행 건물이 모두 완성된 이후에만 건설을 시작할 수 있다.
- 건물 1을 완성해야 건물 2와 건물 3을 건설할 수 있다.
- 건물 2와 건물 3이 모두 완성되어야 건물 4를 건설할 수 있다.
- 각 건물의 건설 시간은 다음과 같다.
 - 건물 1: 10초 / 건물 2: 20초 / 건물 3: 30초 / 건물 4: 40초
- 엘리스는 한 번에 하나의 건물만 건설할 수 없고, 여러 건물을 동시에 건설할 수 있다고 가정한다.

→ 문제1: 위의 조건을 만족하는 위상 정렬 순서는?



위상 정렬 예시



작업순서(위상 정렬)

- $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$
- $1 \rightarrow 3 \rightarrow 2 \rightarrow 4$



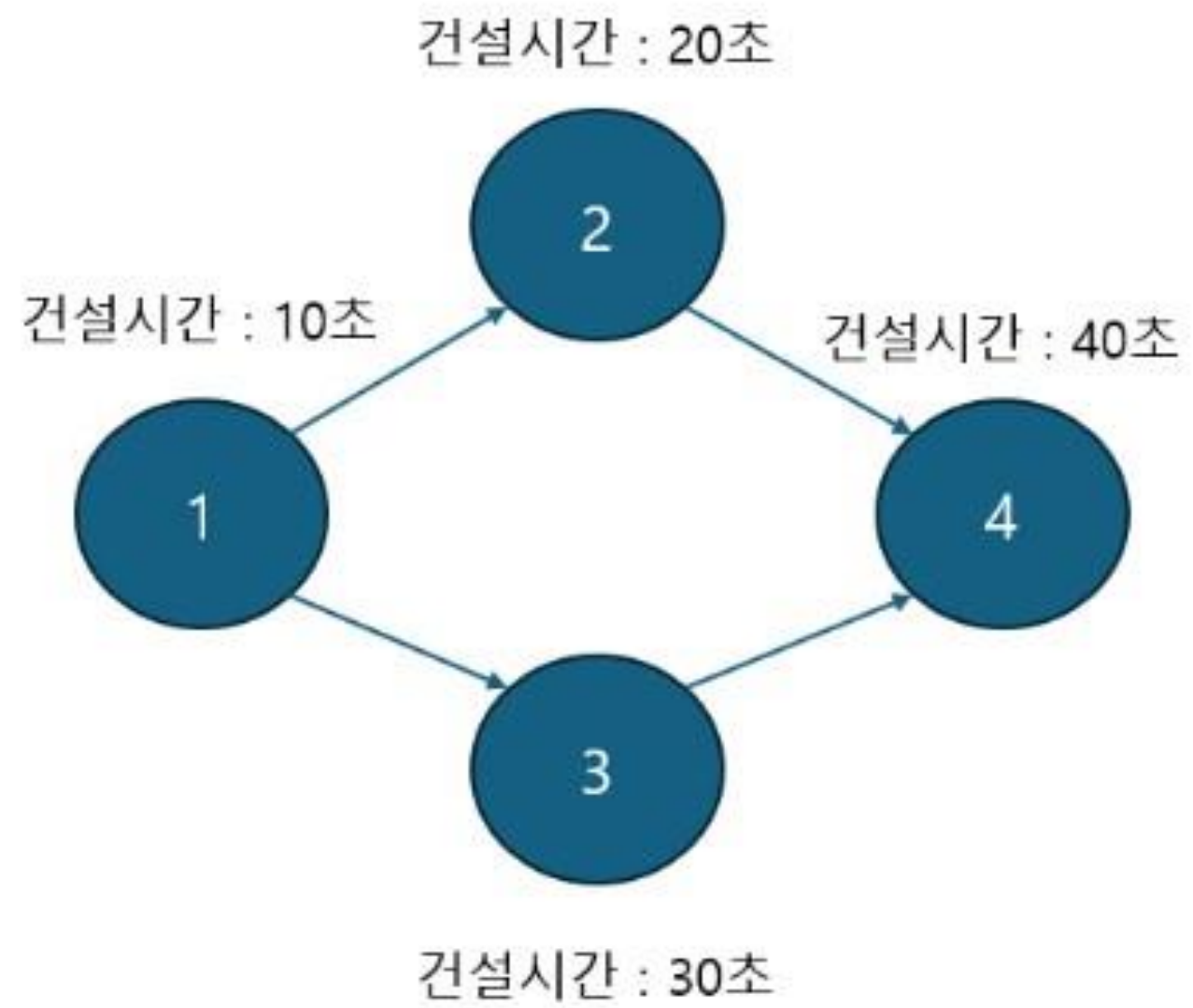
위상 정렬 예시

- 엘리스는 여러 개의 건물을 건설하는 프로젝트를 관리하고 있다.
- 각 건물은 건설을 시작하기 전에 반드시 먼저 완성되어야 하는 건물들이 존재할 수 있다.
- 각 건물은 건설 시간이 서로 다르며, 선행 건물이 모두 완성된 이후에만 건설을 시작할 수 있다.
- 건물 1을 완성해야 건물 2와 건물 3을 건설할 수 있다.
- 건물 2와 건물 3이 모두 완성되어야 건물 4를 건설할 수 있다.
- 각 건물의 건설 시간은 다음과 같다.
 - 건물 1: 10초 / 건물 2: 20초 / 건물 3: 30초 / 건물 4: 40초
- 엘리스는 한 번에 하나의 건물만 건설할 수 없고, 여러 건물을 동시에 건설할 수 있다고 가정한다.

→ 문제2: 모든 건물을 완성하는 데 필요한 최소 시간은?



위상 정렬 예시

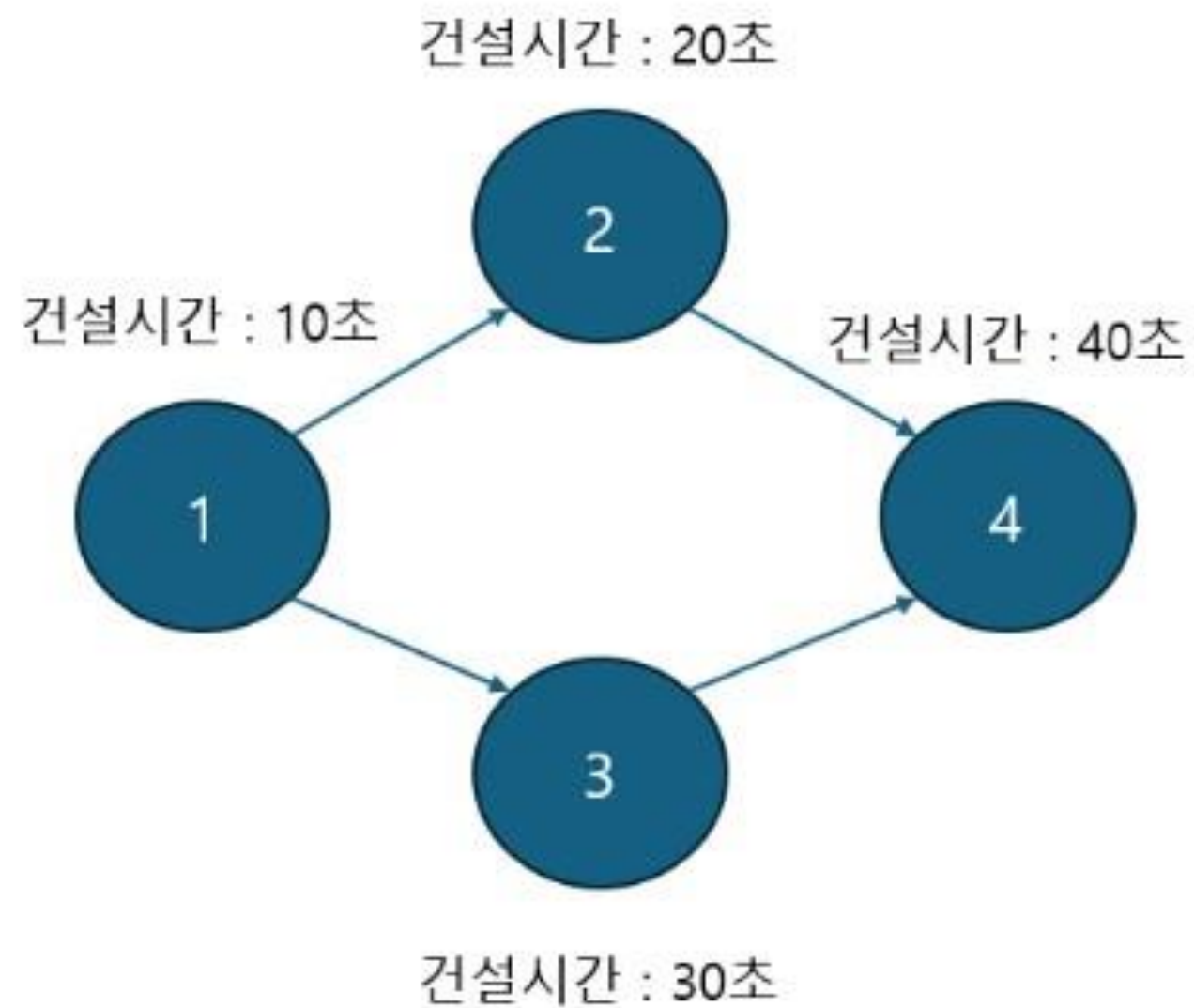


건설시간(최소완료시간)

- 1번 건물 : 10초
- 2번 건물 : 10초 + 20초
- 3번 건물 : 10초 + 30초
- **4번 건물 : ?**



위상 정렬 예시



건설시간(최소완료시간)

- 1번 건물 : 10초
- 2번 건물 : 10초 + 20초
- 3번 건물 : 10초 + 30초
- 4번 건물 : $\max(2\text{번 건물}, 3\text{번 건물}) + 40\text{초}$
- 1, 2, 3, 4번 건물 전체: 80초



1. 진입 차수(In-degree)가 0인 정점을 큐에 삽입

```
for _ in range(k):  
    a, b = map(int,  
input().split())  
    v[a].append(b)  
    degree[b] += 1  
  
for i in range(1, n + 1):  
    if degree[i] == 0:  
        q.append(i)  
        total_time[i] = arr[i]
```



위상 정렬의 구현

2. 큐에서 원소를 꺼내 연결된 모든 간선을 제거
3. 큐가 빌 때까지 1~2번 과정을 반복
 - 모든 원소를 방문하기 전에 큐가 빈다면, 사이클 존재
 - 모든 원소를 방문했다면, 큐에서 꺼낸 순서가 위상 정렬의 결과

```
while q:
    now = q.popleft()
    if now == en:
        break
    for next in v[now]:
        degree[next] -= 1
        if degree[next] == 0:
            q.append(next)

    total_time[next] =
max(total_time[next], total_time[now]
+ arr[next])

print(total_time[en])
```

최소 공통 조상

최소 공통 조상이란?

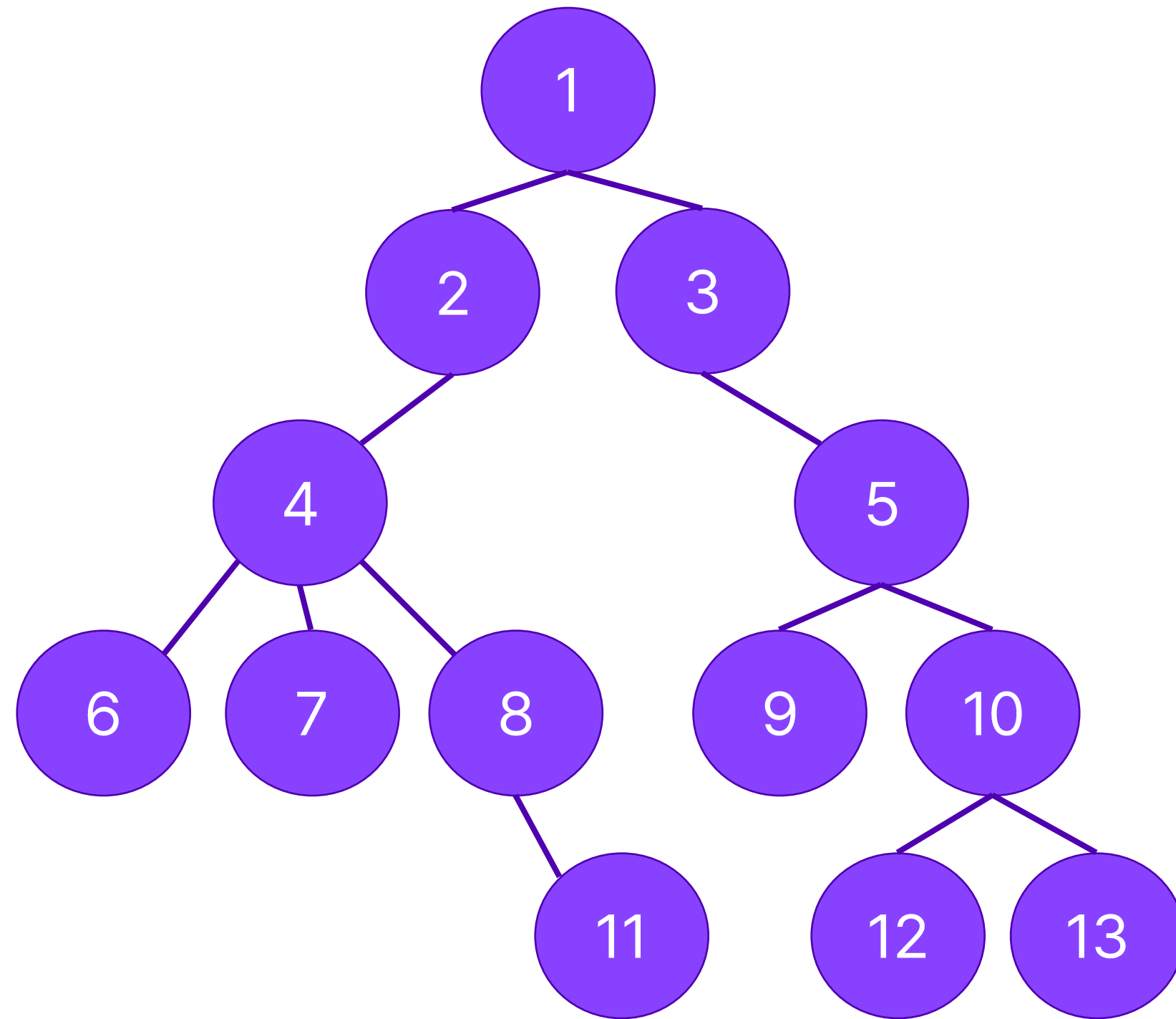
- 두 노드 a, b 에 대하여 a, b 의 가장 가까운 공통 조상을 a, b 의 LCA라고 함

- LCA = Least Common Ancestor

- 7과 10의 LCA는 1

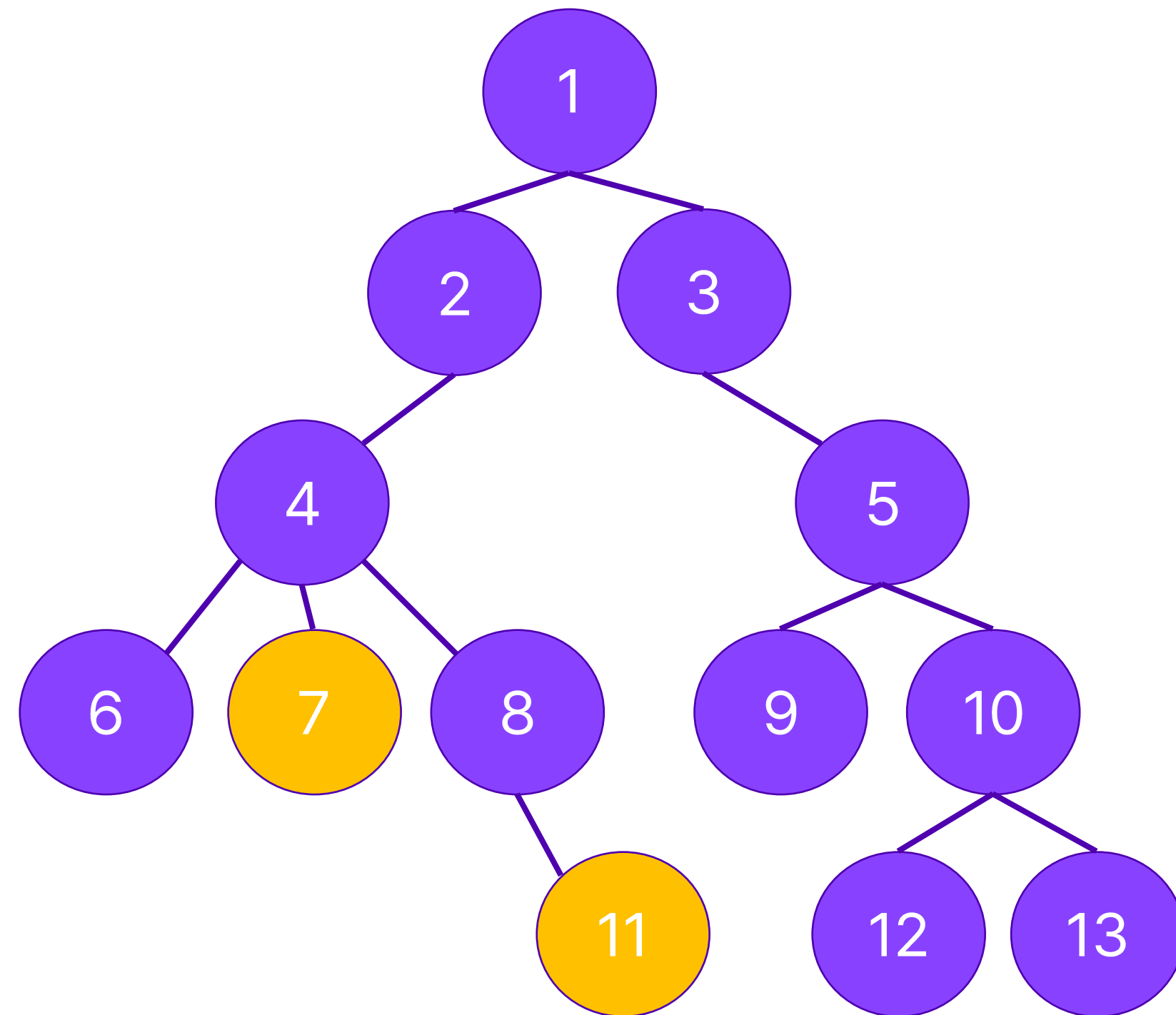
- 5과 9의 LCA는 5

- 6과 11의 LCA는 4



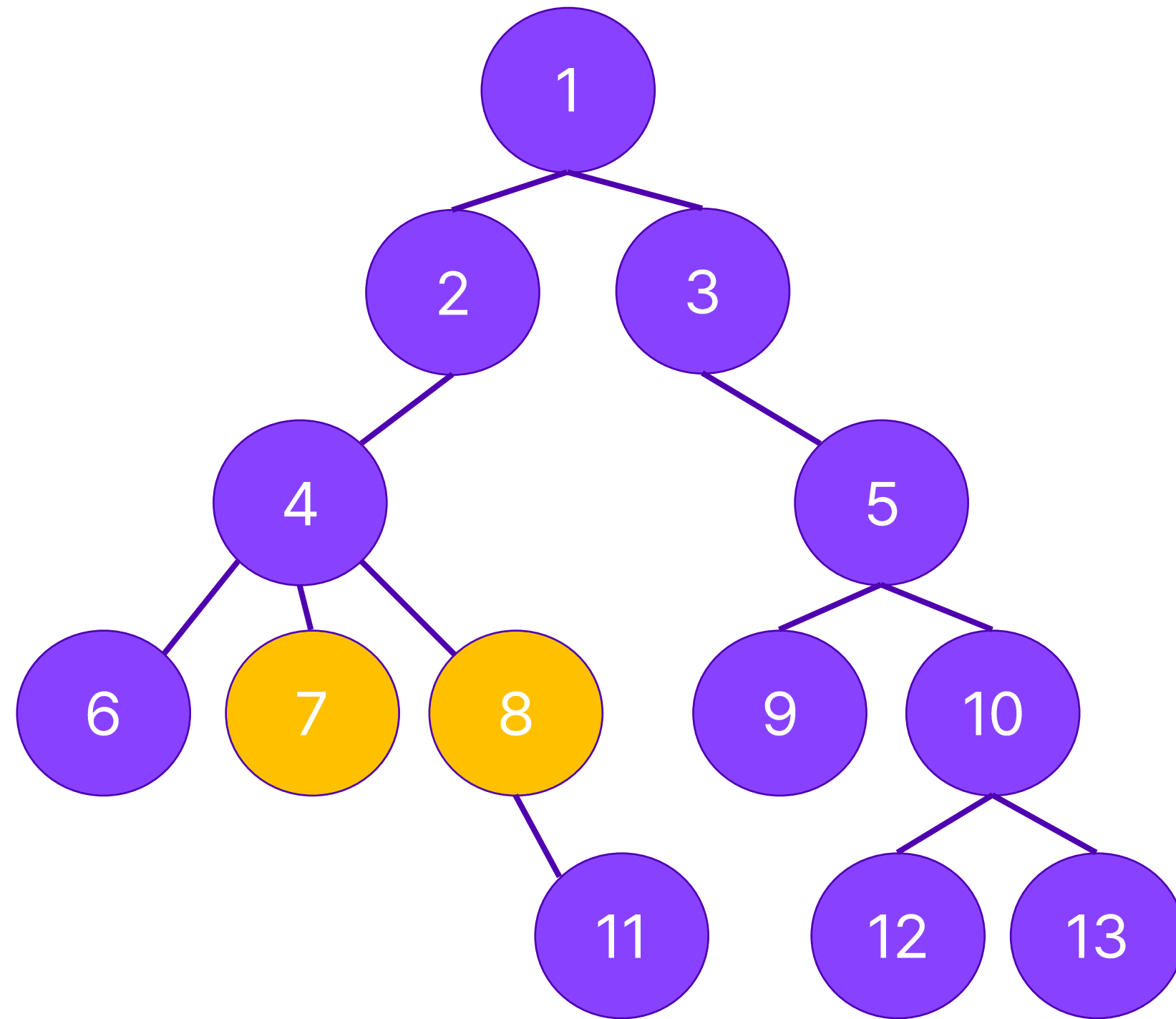
- [illegible]

- 7과 11의 LCA는?



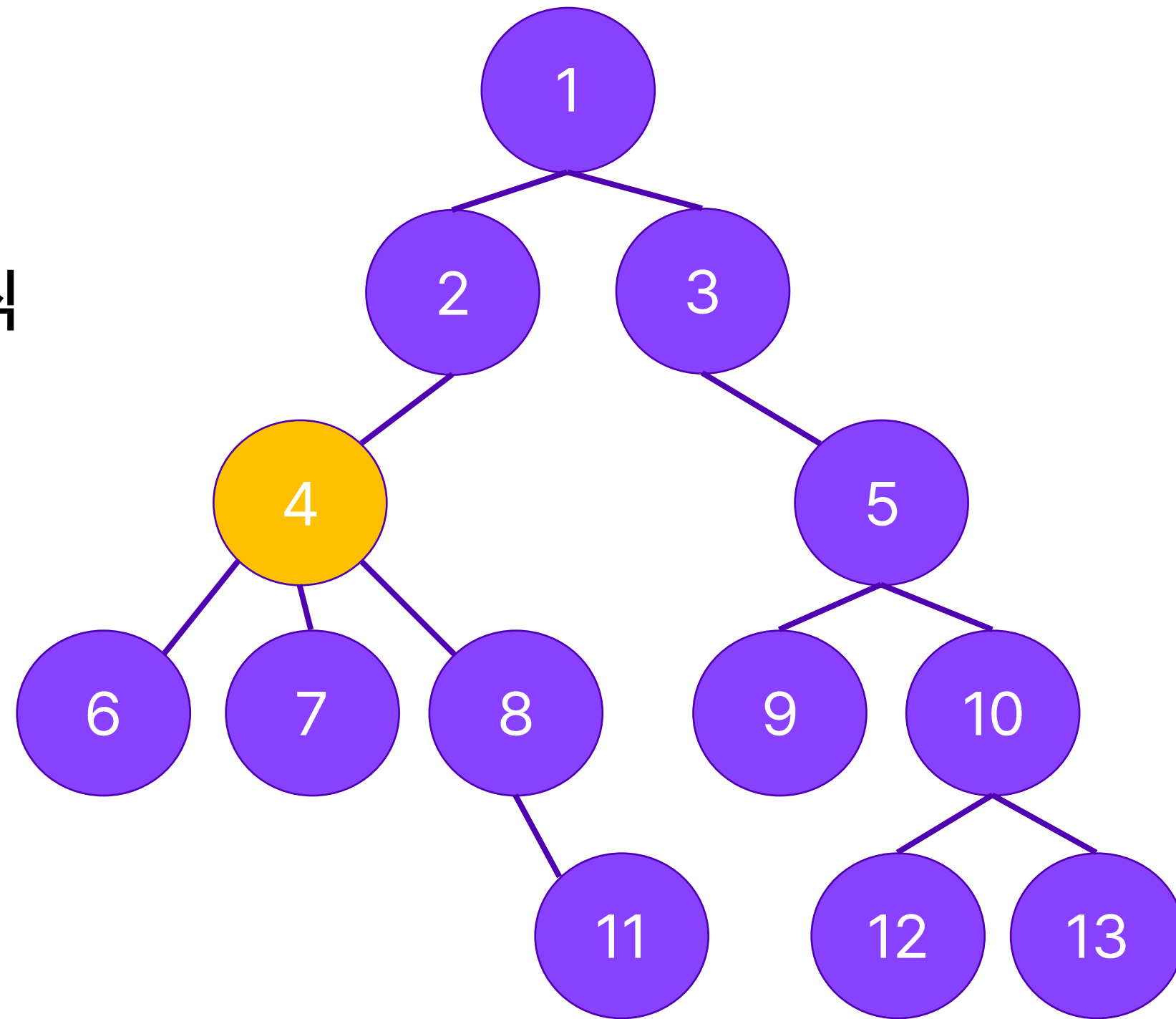
- 7과 11의 LCA는?

- 11을 하나씩 올려서 7의 깊이와 같도록 맞추어
 쭈м

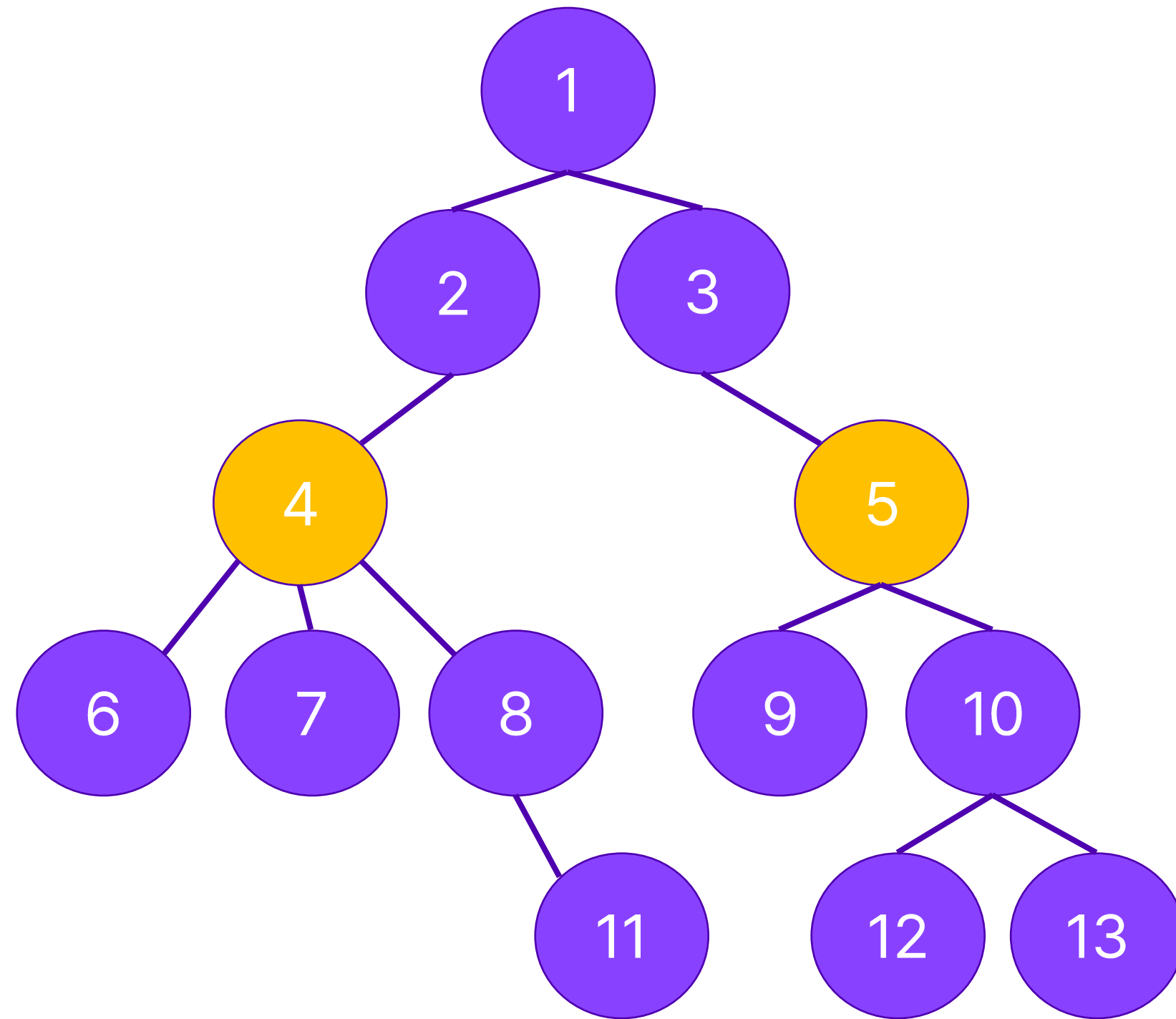


• 7과 11의 LCA는?

- 11을 하나씩 올려서 7의 깊이와 같도록 맞추어
 줄
- 이제, 같은 노드를 가리킬 때까지 동시에 하나씩
 올려줄
- 이때, 4에서 멈추게 됨

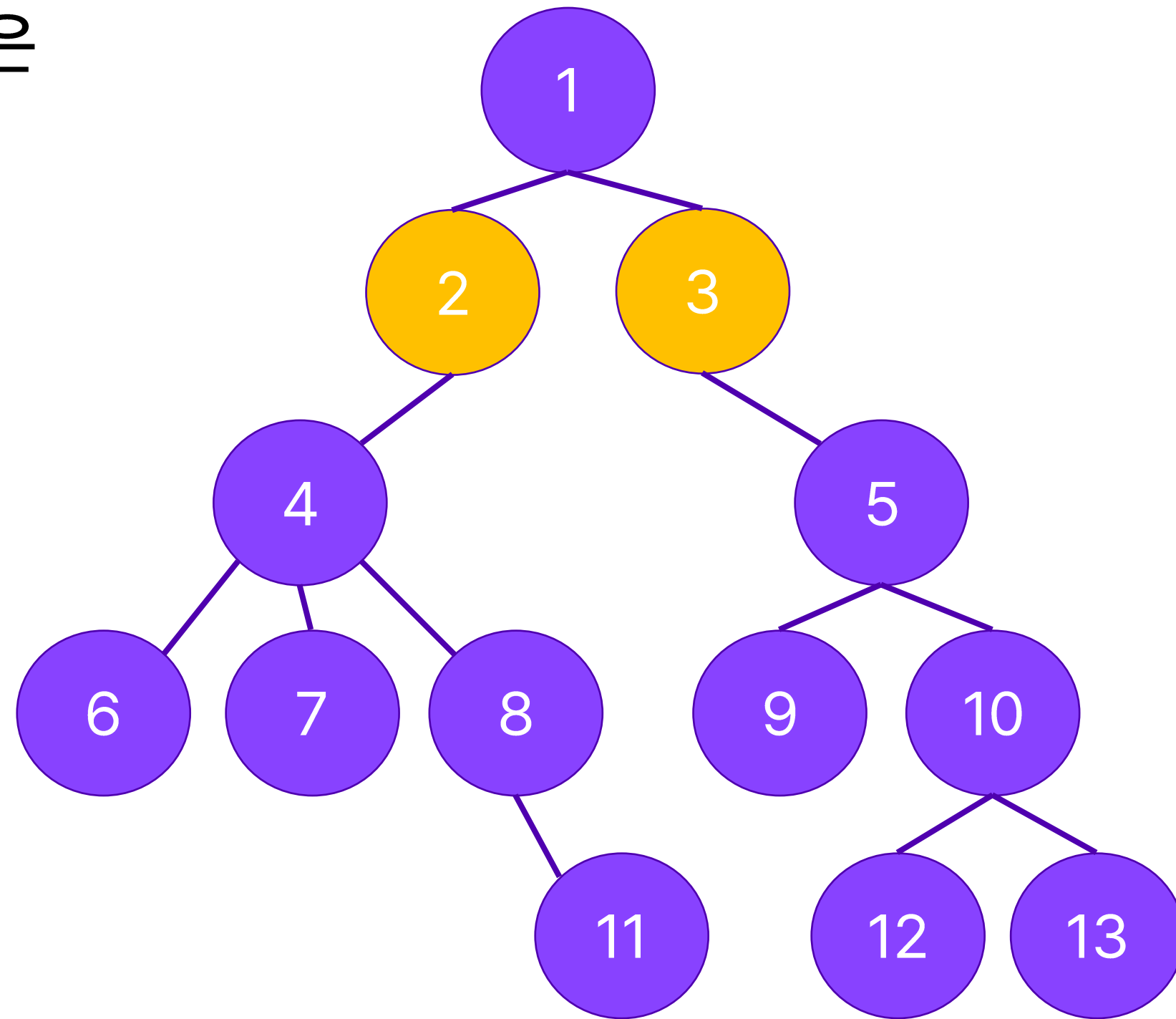


• 4와 5의 LCA는?



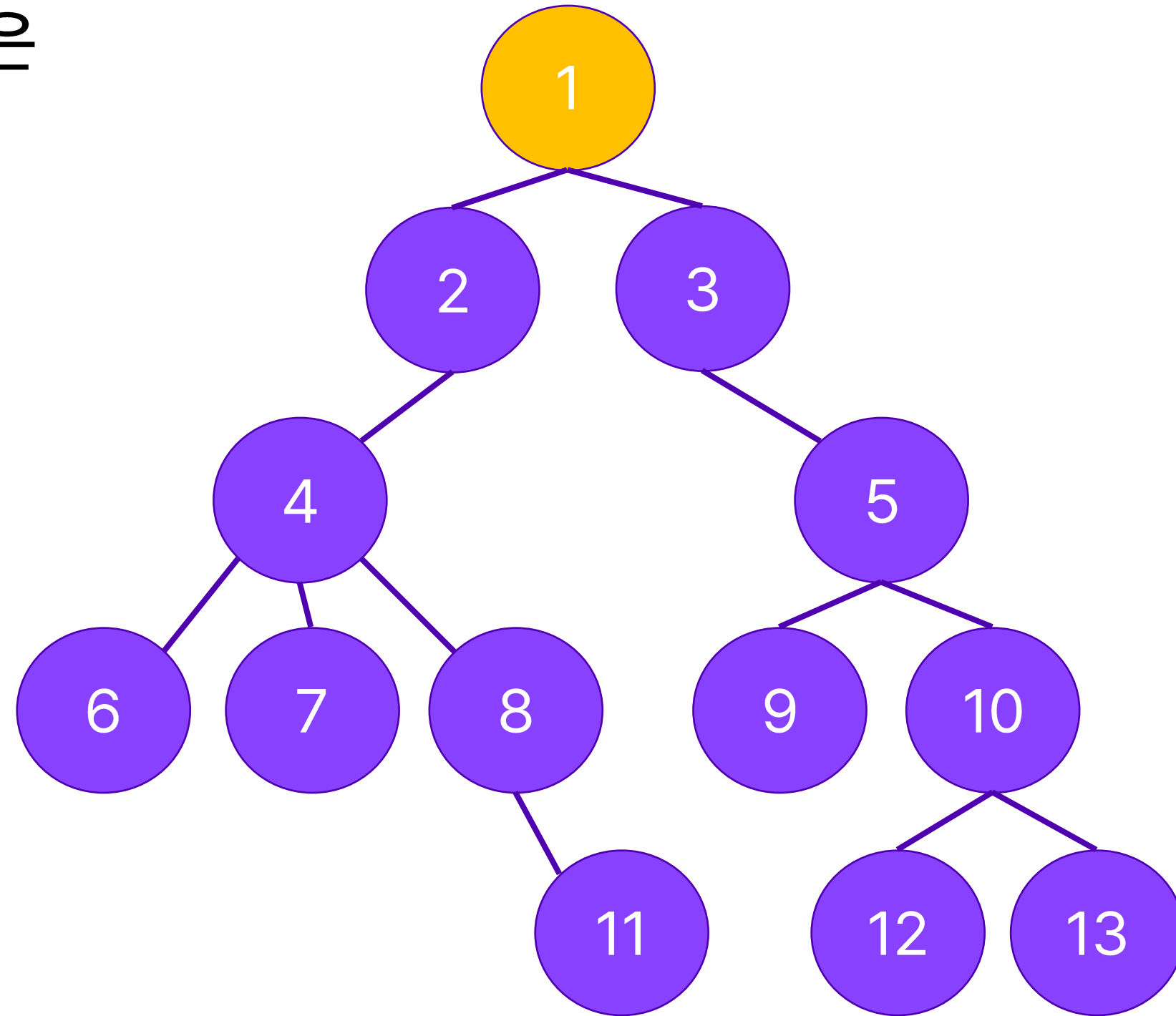
- 4와 5의 LCA는?

- 깊이는 이미 같으므로 바로, 두 노드가 서로 같은 노드를 가리킬 때까지 동시에 하나씩 올려줌



• 4와 5의 LCA는?

- 깊이는 이미 같으므로 바로, 두 노드가 서로 같은 노드를 가리킬 때까지 동시에 하나씩 올려줌
- 노드 1에서 종료





1. a 가 b 보다 깊이가 크도록 설정
 - depth 배열에는 깊이 저장
 - $\text{swap}(a, b) \rightarrow a$ 와 b 의 값을 서로 바꿈
2. a 의 깊이를 b 의 깊이와 같게끔 하나씩 올려줌
3. a 와 b 의 깊이가 같아졌다면, 이후부터는 a 와 b 를 동시에 하나씩 올려줌
4. a 와 b 가 같은 노드를 가리키게 되면 종료하게 되고, 해당 노드를 return

```
def lca(a, b, parent, depth) :  
    # depth[a] >= depth[b]가 되도록  
    정렬  
    if depth[a] < depth[b]:  
        a, b = b, a  
  
    # 깊이 맞추기  
    while depth[a] != depth[b]:  
        a = parent[a]  
  
    # 같은 노드가 될 때까지 동시에 올리기  
    while a != b:  
        a = parent[a]  
        b = parent[b]  
  
    return a
```